

MPI sobre clúster AWS: multiplicación de matrices con paso de mensajes

Caso de Estudio 3 — Computación de Alto Rendimiento

Daniel Toro Soto , Juan Camilo Galvis

Mayo 2026

Resumen

El presente documento describe la implementación, despliegue y análisis de desempeño de una versión paralela con MPI (*Message Passing Interface*) del algoritmo de multiplicación de matrices cuadradas en lenguaje C, ejecutada sobre un clúster real desplegado con AWS ParallelCluster. La infraestructura consta de un nodo *HeadNode* de orquestación y tres nodos de cómputo *c7i-flex.large* (2 vCPUs y 8 GB de RAM por nodo, para un total de 6 vCPUs distribuidas), administrados por Slurm y comunicados a través de un sistema de archivos compartido EFS montado por NFS. Se ejecutaron 11 tamaños de matriz (entre 504 y 4008) con 7 repeticiones por configuración para tres niveles de paralelismo (2, 4 y 6 procesos MPI), tomando como línea base un binario secuencial monolítico. Los resultados muestran que la implementación punto a punto escala de manera consistente al aumentar el número de procesos y que el clúster alcanza speedups cercanos a $8,7\times$ para matrices de 4008×4008 usando los 6 procesos disponibles.

Índice

1. Introducción	3
2. Marco Conceptual	3
2.1. High Performance Computing (HPC)	3
2.2. Multiplicación matricial	3
2.3. Computación distribuida vs. memoria compartida	3
2.4. MPI: Message Passing Interface	3
2.5. Comunicación punto a punto	4
2.6. Comunicación colectiva	4
2.7. Cluster middleware	4
2.8. Speedup	4
3. Marco Contextual	5
3.1. Infraestructura del clúster en AWS	5
3.2. Pila de software	5
3.3. Sistema de compilación	6
3.4. Algoritmo MPI punto a punto	6
3.5. Sistema de benchmarking	7
4. Despliegue del clúster	7
4.1. Creación del clúster	7
4.2. Conexión al HeadNode	8
4.3. Transferencia del código al EFS compartido	8
4.4. Asignación de recursos con Slurm	9
4.5. Ejecución del benchmark	9

5. Pruebas	10
5.1. Secuencial monolítico	10
5.2. MPI Point-to-Point — 2 procesos	10
5.3. MPI Point-to-Point — 4 procesos	11
5.4. MPI Point-to-Point — 6 procesos	11
6. Gráficas comparativas	11
6.1. Tiempo de ejecución	11
6.2. Speedup	12
6.3. Heatmap de speedup	12
7. Análisis de Resultados	13
7.1. Comportamiento del speedup observado	13
7.2. Picos en matrices pequeñas	13
7.3. Impacto del overhead de comunicación	13
7.4. Configuración de Slurm y techo de paralelismo	13
7.5. Comparación cualitativa con paralelismo de memoria compartida	13
8. Conclusiones	14
9. Bibliografía	15

1. Introducción

La computación de alto rendimiento (HPC) busca resolver problemas de gran escala distribuyendo el trabajo entre múltiples recursos de cómputo. En casos de estudio previos del curso, la multiplicación de matrices cuadradas se paralelizó sobre una sola máquina, aprovechando memoria compartida mediante hilos POSIX y procesos creados con `fork()`. En este caso de estudio se da un paso adicional: la paralelización se realiza sobre memoria *distribuida*, donde los procesos no comparten espacio de direcciones y deben comunicarse explícitamente por la red mediante un estándar de paso de mensajes.

El estándar utilizado es MPI (*Message Passing Interface*), específicamente la implementación OpenMPI provista por la distribución Amazon Linux 2 del clúster. El despliegue de la infraestructura se realiza con `aws-parallelcluster`, una herramienta oficial de AWS que abstrae la creación de un *HeadNode*, los nodos de cómputo y el almacenamiento compartido EFS necesarios para ejecutar trabajos paralelos sobre Slurm. El enunciado del caso de estudio exige un mínimo de 3 nodos de cómputo y advierte explícitamente que la medición de tiempo no debe usar relojes de pared locales en cada nodo, sino el mecanismo propio de MPI (`MPI_Wtime`).

El objetivo de este informe es: (i) documentar la infraestructura del clúster y el proceso de despliegue; (ii) describir el algoritmo MPI punto a punto implementado; (iii) reportar los tiempos de ejecución y speedups obtenidos para 11 dimensiones de matriz y 3 niveles de paralelismo; y (iv) analizar el comportamiento de escalabilidad observado al variar el número de procesos.

2. Marco Conceptual

2.1. High Performance Computing (HPC)

Cuando hablamos de HPC “hacemos referencia a un campo de la computación actual que da solución a problemas tecnológicos muy complejos y que involucran un gran volumen de cálculos o de coste computacional” (López, 2017)[3]. El producto matricial es un problema de referencia para evaluar HPC porque su complejidad cúbica crece muy rápido con la dimensión de la matriz, lo que justifica explorar estrategias de paralelización tanto en memoria compartida como en memoria distribuida.

2.2. Multiplicación matricial

Dadas dos matrices cuadradas A y B de dimensión n :

$$A := (a_{ij})_{n \times n} \quad B := (b_{ij})_{n \times n}$$

el producto $C = AB$ genera una matriz $C := (c_{ij})_{n \times n}$ donde

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}.$$

El algoritmo ingenuo consiste en tres bucles anidados, con complejidad temporal $\mathcal{O}(n^3)$ y espacial $\mathcal{O}(n^2)$ (Cortéz, 2004)[1].

2.3. Computación distribuida vs. memoria compartida

En memoria compartida, los hilos o procesos viven dentro de una misma máquina y comparten un mismo espacio de direcciones (o bien lo simulan vía `mmap`). El costo de comunicación entre trabajadores es bajo: basta con escribir y leer la misma región de memoria. En memoria distribuida, cada proceso vive en una máquina distinta y la única forma de coordinarse es enviar mensajes a través de la red. Esto introduce un *overhead* de comunicación que no existía en los casos anteriores, pero a cambio permite escalar el problema más allá del límite de una sola máquina (Ferestrepoca, 2019)[2].

2.4. MPI: Message Passing Interface

MPI es el estándar *de facto* para programación paralela en memoria distribuida. Define una API en C/Fortran/Python que abstrae el paso de mensajes entre procesos independientes, y oculta los detalles del medio de transporte subyacente (TCP, UDP, InfiniBand, etc.). Sus elementos fundamentales son:

- **Communicator:** grupo lógico de procesos que pueden intercambiar mensajes. El comunicador por defecto es `MPI_COMM_WORLD` y agrupa a todos los procesos lanzados por `mpirun`.
- **Rank:** identificador entero único de cada proceso dentro de un comunicador. El rank 0 suele actuar como “maestro” que orquesta el trabajo.
- **Tag:** número entero asociado a cada mensaje que permite distinguir contenidos distintos enviados entre el mismo par de procesos.

En este caso de estudio se utiliza la versión 5.0 del estándar (MPI Forum)[5] y la implementación OpenMPI[6].

2.5. Comunicación punto a punto

La forma más básica de intercambio en MPI es la comunicación punto a punto entre dos procesos. La primitiva `MPI_Send(buf, count, datatype, dest, tag, comm)` envía `count` elementos del tipo `datatype` desde `buf` hacia el proceso `dest`, con el identificador `tag` dentro del comunicador `comm`. La contraparte `MPI_Recv(buf, count, datatype, source, tag, comm, status)` bloquea hasta recibir un mensaje compatible. Esta primitiva es suficiente para implementar un patrón maestro-esclavo donde el rank 0 reparte trabajo y recoge resultados.

2.6. Comunicación colectiva

MPI también ofrece operaciones colectivas, en las que la llamada se hace sobre todo un comunicador y se traduce internamente en múltiples envíos optimizados. Entre las más relevantes:

- **MPI_Scatter:** parte un buffer del root en *chunks* y entrega un chunk a cada proceso del comunicador.
- **MPI_Gather:** operación inversa, recoge los chunks de todos los procesos en el root.
- **MPI_Reduce / MPI_Allreduce:** aplican una operación de reducción (suma, máximo, etc.) sobre datos distribuidos; **Allreduce** además distribuye el resultado a todos los procesos.
- **MPI_Barrier:** sincronización; ningún proceso avanza hasta que todos llegan a la barrera.

En este informe la implementación medida es la versión punto a punto; el repositorio también contiene una versión colectiva (`src/CollectiveMatrixSolver.c`) como referencia.

2.7. Cluster middleware

Para que un conjunto de máquinas se comporte como un único clúster se requieren tres piezas de *middleware*: un **programa de despliegue** (en este caso `aws-parallelcluster`), un **scheduler** que reparta trabajos sobre los nodos (**Slurm**) y un **sistema de archivos compartido** que evite tener que copiar binarios a cada nodo (**EFS** montado por NFS en `/shared`). Slurm provee comandos como `sinfo` (estado de nodos), `salloc` (solicitar asignación interactiva) y `srun / mpirun` (lanzar el trabajo paralelo).

2.8. Speedup

El speedup representa la ganancia en rendimiento de la versión paralela respecto a la secuencial:

$$S_p = \frac{T^*(n)}{T_p(n)},$$

donde $T^*(n)$ es el tiempo de la implementación secuencial monolítica para una matriz de dimensión n y $T_p(n)$ el tiempo de la implementación MPI con p procesos. En este caso de estudio se toma como línea base el binario secuencial corriendo en un único nodo de cómputo, tal como permite el enunciado. El speedup ideal es lineal ($S_p = p$), pero en la práctica se ve limitado por porciones secuenciales del algoritmo (Ley de Amdahl)[4] y por el overhead de comunicación inherente a la memoria distribuida.

3. Marco Contextual

3.1. Infraestructura del clúster en AWS

El clúster se desplegó en la región `us-east-1` de AWS usando `aws-parallelcluster`. Los nodos se ubican en la misma subred (`subnet-0bd01cb1a60e9790a`) para garantizar latencias bajas dentro de la misma zona de disponibilidad. La configuración utilizada es:

- **HeadNode:** instancia `t3.micro` con 2 vCPUs y 1 GB de RAM. Su rol es exclusivamente de orquestación: alberga el scheduler Slurm, compila el código y lanza los trabajos. Por su limitado tamaño no participa en el cómputo de las pruebas.
- **Nodos de cómputo:** 3 instancias `c7i-flex.large` con 2 vCPUs y 8 GB de RAM cada una, para un total de **6 vCPUs y 24 GB de RAM agregados**. La cola Slurm se llama `compute`, con `MinCount=3` de modo que los tres nodos están siempre activos (requisito del enunciado de tener al menos 3 nodos de cómputo).
- **Sistema operativo:** Amazon Linux 2, con OpenMPI disponible por defecto.
- **Almacenamiento compartido:** EFS montado en `/shared` (vía NFS) en el HeadNode y en cada nodo de cómputo. Esto elimina la necesidad de copiar el binario y los scripts a cada nodo: basta con compilar una sola vez en el HeadNode para que todos los nodos vean el ejecutable.

El YAML de configuración entregado a `pcluster create-cluster` es el siguiente:

```
Region: us-east-1
Image:
  Os: alinux2

HeadNode:
  InstanceType: t3.micro
  Networking:
    SubnetId: subnet-0bd01cb1a60e9790a
  Ssh:
    KeyName: hpc-mpi-key

Scheduling:
  Scheduler: slurm
  SlurmQueues:
    - Name: compute
      ComputeResources:
        - Name: c5xlarge
          InstanceType: c7i-flex.large
          MinCount: 3
          MaxCount: 4
      Networking:
        SubnetIds:
          - subnet-0bd01cb1a60e9790a

SharedStorage:
  - MountDir: /shared
    Name: nfs-shared
    StorageType: Efs
    EfsSettings:
      Encrypted: false
```

3.2. Pila de software

Para desplegar y operar el clúster desde la máquina local se requirió la siguiente pila:

- **AWS CLI v2:** herramienta oficial para autenticarse contra AWS y obtener metadatos (IPs de instancias, subredes disponibles, etc.).

- **AWS ParallelCluster CLI:** paquete `aws-parallelcluster` instalado con `pip3`. **Importante:** en el momento del desarrollo el paquete sólo era compatible con **Python 3.9**; versiones más recientes de Python causaron problemas de dependencias, por lo que fue necesario instalar específicamente esa versión.
- **OpenMPI:** proporcionado por la imagen Amazon Linux 2 del HeadNode y los nodos de cómputo. Provee tanto el compilador `mpicc` como el lanzador `mpirun`.
- **GCC:** usado para compilar el binario secuencial de referencia.

El código fuente del proyecto se encuentra en: <https://github.com/pausa11/HPC>

3.3. Sistema de compilación

El `Makefile` del proyecto produce tres binarios en el directorio `output/`:

```
$(OUT)/secuential: src/SequentialMatrixSolver.c
    gcc -o $@ $<

$(OUT)/point_to_point: src/PointToPoint.c
    mpicc -o $@ -O2 $<

$(OUT)/collective: src/CollectiveMatrixSolver.c
    mpicc -o $@ -O2 $<
```

El binario `secuential` no recibe argumentos especiales del compilador, mientras que las variantes MPI usan `mpicc` (alias de `gcc` con las librerías de OpenMPI ya enlazadas) con `-O2` para activar las optimizaciones automáticas estándar (vectorización SIMD, reordenamiento de instrucciones, etc.).

3.4. Algoritmo MPI punto a punto

La implementación de `src/PointToPoint.c` sigue un patrón maestro-esclavo clásico:

1. **Inicialización:** todos los procesos llaman a `MPI_Init` y consultan su `rank` y el tamaño del comunicador. Argumentos: dimensión de la matriz y número total de procesos.
2. **Validación:** la dimensión de la matriz debe ser divisible por el número de procesos para repartir filas en partes iguales. Por eso los tamaños evaluados son múltiplos de 12 (504, 1008, 1296, 1608, ..., 4008).
3. **Rank 0 (maestro):** asigna y llena las matrices A y B con enteros aleatorios entre 0 y 9. Toma `MPI_Wtime()` como punto de inicio. Para cada worker $i = 1, \dots, p - 1$ envía:
 - Un *batch* de filas de A (tag 0).
 - La matriz B completa (tag 1), necesaria porque cada worker debe poder acceder a cualquier columna de B al calcular su porción de C .

Luego el rank 0 calcula localmente el último batch de filas (las que le corresponderían como “worker”), y finalmente hace `MPI_Recv` para recoger los batches de C de cada worker (tag 2). Al recibir el último mensaje toma `MPI_Wtime()` como punto final y reporta la diferencia.

4. **Workers (rank $\neq 0$):** reciben su batch de A y la matriz B , calculan su porción de C con el algoritmo ingenuo de tres bucles anidados, y devuelven el resultado al rank 0.
5. **Finalización:** todos los procesos llaman a `MPI_Finalize`.

El uso de `MPI_Wtime` resuelve el problema mencionado en el enunciado sobre relojes heterogéneos entre nodos: la medición se hace siempre sobre el mismo reloj (el del rank 0) y representa el tiempo de pared transcurrido desde que comienza la generación de datos hasta que se recibe el último batch de C .

3.5. Sistema de benchmarking

El script `scripts/RunAll.sh` ejecuta el barrido completo de pruebas con un sistema de checkpoints similar al de los casos de estudio anteriores: cada combinación (tamaño, número de procesos, repetición) que termina con éxito se registra en `checkpoint.log`, de modo que si la corrida se interrumpe puede reanudarse sin repetir trabajo ya hecho. El bucle interno itera sobre los tamaños y guarda los tiempos en archivos CSV separados por cantidad de procesos:

```
sizes=(504 1008 1296 1608 2004 2304 2604 3000 3300 3600 4008)
num_processes=(2 4 6)

for n in "${num_processes[@]"; do
  for j in $(seq 1 10); do
    for i in "${sizes[@]"; do
      mpirun -n "$n" "$ROOT_DIR/output/point_to_point" "$i" "$n"
    done
  done
done
```

Como la corrida completa toma varias horas y la sesión SSH puede caerse, en la práctica se lanzó el script en segundo plano con `nohup`:

```
nohup bash RunAll.sh > runall.log 2>&1 &
```

La corrida final reportada en este informe contiene **7 repeticiones** por configuración (en lugar de 10) debido a los tiempos de ejecución acumulados sobre las instancias `c7i-flex.large`.

4. Despliegue del clúster

A continuación se documenta el flujo real ejecutado para llevar el clúster a un estado funcional. Cada subsección incluye una captura del comando ejecutado y su salida en el terminal.

4.1. Creación del clúster

Una vez configuradas las credenciales de AWS, el par de claves SSH y el archivo YAML descrito en la sección 3.1, el clúster se desplegó con:

```
pcluster create-cluster \
  --cluster-name hpc-mpi-cluster \
  --cluster-configuration ~/hpc-cluster-config.yaml
```

El comando dispara internamente la creación de un stack de CloudFormation con todas las instancias EC2, los grupos de seguridad, el EFS y la configuración Slurm. La creación pasa por los estados `CREATE_IN_PROGRESS` y termina en `CREATE_COMPLETE` tras aproximadamente 10–15 minutos.


```
scp -i ~/.ssh/hpc-mpi-key.pem -r \
/Users/danieltorosoto/universidad/HPC/caso-estudio-3 \
ec2-user@3.235.161.247:/shared/caso-estudio-3
```

```

○ danieltorosoto@Mac-mini-de-Daniel HPC % source /Users/danieltorosoto/universidad/HPC/.venv/bin/activate
(.venv) danieltorosoto@Mac-mini-de-Daniel HPC % scp -i ~/.ssh/hpc-mpi-key.pem -r \
/Users/danieltorosoto/universidad/HPC/caso-estudio-3 \
ec2-user@3.235.161.247:/shared/caso-estudio-3
deploying_parallelcluster.png      100% 217KB 498.0KB/s   00:00
Makefile                          100% 292    4.0KB/s   00:00
enunciado.md                      100% 940   12.3KB/s   00:00
apuntes.md                        100% 6891  89.0KB/s   00:00
plan.md                           100% 6131  81.0KB/s   00:00
RunAll.sh                         100% 3153  42.7KB/s   00:00
hpc-cluster-config.yaml           100% 610    8.2KB/s   00:00
SequentialMatrixSolver.c           100% 4107  55.3KB/s   00:00
PointToPoint.c                    100% 5866  77.2KB/s   00:00
(.venv) danieltorosoto@Mac-mini-de-Daniel HPC %

```

Figura 3: Transferencia del proyecto al directorio compartido `/shared` del HeadNode.

4.4. Asignación de recursos con Slurm

Tras compilar el proyecto con `make`, se solicitó una asignación interactiva de recursos a Slurm. El comando ejecutado en la práctica fue:

```
salloc --nodes=3 --ntasks=6 --ntasks-per-node=2 --partition=compute
```

Esto reserva los 3 nodos de cómputo simultáneamente, con 2 tareas por nodo, para un total de 6 procesos MPI disponibles (límite natural impuesto por la configuración: 3 nodos $\times$ 2 vCPUs/nodo = 6 vCPUs). Con esta asignación abierta, todas las llamadas subsiguientes a `mpirun` se distribuyen sobre los nodos reservados.

```

[ec2-user@ip-172-31-10-216 shared]$ salloc --nodes=3 --ntasks=6 --ntasks-per-node=2 --partition=compute
salloc: Granted job allocation 3
salloc: Nodes compute-st-c5xlarge-[1-3] are ready for job
[ec2-user@ip-172-31-10-216 shared]$

```

Figura 4: Asignación de recursos con `salloc` para 3 nodos y 6 tareas.

4.5. Ejecución del benchmark

Por último, el barrido completo de pruebas se lanzó en segundo plano con `nohup`, lo que permite que la sesión SSH pueda cerrarse sin abortar el proceso:

```
nohup bash RunAll.sh > runall.log 2>&1 &
```

```

[ec2-user@ip-172-31-10-216 caso-estudio-3]$ cd scripts/
[ec2-user@ip-172-31-10-216 scripts]$ bash RunAll.sh
[NEW] Iniciando nueva corrida: /shared/caso-estudio-3/stats/ip-172-31-10-216/20260525_021456
Point to Point testing in process ...
Point to Point for 2 testing in process ...
Warning: Permanently added 'compute-st-c5xlarge-1,172.31.2.196' (ECDSA) to the list of known ho
sts.
Warning: Permanently added 'compute-st-c5xlarge-2,172.31.13.213' (ECDSA) to the list of known h
osts.
Warning: Permanently added 'compute-st-c5xlarge-3,172.31.10.6' (ECDSA) to the list of known hos
ts.
Warning: Permanently added 'compute-st-c5xlarge-1,172.31.2.196' (ECDSA) to the list of known ho
sts.
Warning: Permanently added 'compute-st-c5xlarge-2,172.31.13.213' (ECDSA) to the list of known h
osts.
Warning: Permanently added 'compute-st-c5xlarge-3,172.31.10.6' (ECDSA) to the list of known hos
ts.
Warning: Permanently added 'compute-st-c5xlarge-2,172.31.13.213' (ECDSA) to the list of known h
osts.
Warning: Permanently added 'compute-st-c5xlarge-1,172.31.2.196' (ECDSA) to the list of known ho
sts.
Warning: Permanently added 'compute-st-c5xlarge-3,172.31.10.6' (ECDSA) to the list of known hos
ts.

```

Figura 5: Ejecución del benchmark en background sobre el clúster.

Al concluir la corrida, los CSV resultantes se recuperaron a la máquina local con:

```

scp -i ~/.ssh/hpc-mpi-key.pem -r \
ec2-user@3.235.161.247:/shared/caso-estudio-3/stats \
/Users/danielatorosoto/universidad/HPC/caso-estudio-3/stats/

```

5. Pruebas

A continuación se presentan los tiempos de ejecución (en segundos) medidos sobre el clúster, con 7 repeticiones por configuración. Los tamaños evaluados son: **504, 1008, 1296, 1608, 2004, 2304, 2604, 3000, 3300, 3600 y 4008**. La fila **Pr** corresponde al promedio sobre las 7 corridas y la fila **Speedup** es la razón $T_{seq}(n)/T_p(n)$ respecto al promedio secuencial de la Tabla 1.

5.1. Secuencial monolítico

Secuencial monolítico (1 nodo)											
Dimensión	504	1008	1296	1608	2004	2304	2604	3000	3300	3600	4008
1	0.8106	12.3060	27.0610	57.3761	95.3988	176.7559	195.9331	389.4349	432.1465	678.6314	945.0613
2	0.8021	12.0140	26.4754	57.2960	96.5203	176.9417	195.6262	389.2148	433.9327	678.9654	944.9075
3	0.8008	11.7629	26.8397	57.7502	95.8755	176.4996	195.7034	392.1800	430.9143	678.8128	945.8382
4	0.8125	11.7242	27.5700	56.6952	95.5283	176.0975	195.9362	390.2106	429.4717	677.3248	945.7674
5	0.7977	11.7082	27.5933	57.0412	96.3679	175.7092	195.7642	389.1404	431.5241	679.0673	944.3342
6	0.7971	12.0546	28.2674	56.3694	94.9194	175.9146	196.4258	389.5462	432.2447	677.3263	941.9913
7	0.7995	12.9266	27.1167	56.6794	95.0250	176.7734	196.0549	390.7890	432.2724	679.6710	945.9243
Pr	0.8029	12.0710	27.2748	57.0297	95.6622	176.3845	195.9205	390.0737	431.7581	678.5427	944.8320

Cuadro 1: Tiempos del binario secuencial monolítico ejecutado en un único nodo del clúster.

5.2. MPI Point-to-Point — 2 procesos

MPI Point-to-Point — 2 procesos											
Dim	504	1008	1296	1608	2004	2304	2604	3000	3300	3600	4008
1	0.0740	1.1404	6.8542	18.3174	34.1801	70.2925	80.0321	130.5400	169.4479	243.8874	325.6519
2	0.0786	1.1559	8.4621	18.2545	36.3381	70.4790	82.0369	133.4042	172.0957	244.6817	326.1514
3	0.0755	1.1292	6.9728	18.5749	36.1755	67.3961	81.9096	131.5066	169.4137	242.1031	325.6040
4	0.0745	1.1796	8.0887	17.4019	36.1874	68.4356	81.5723	131.4370	169.9215	242.1586	319.4499
5	0.0747	1.1743	8.3806	18.8673	35.1995	69.6396	80.4419	123.5418	169.6436	242.8427	322.6000
6	0.0782	1.1888	8.4785	18.4682	35.1357	67.6840	80.3217	129.4407	171.8913	244.0642	322.6987
7	0.0770	1.2002	8.2369	18.5228	35.1443	70.5051	81.1060	132.2446	171.6062	244.4162	326.4506
Pr	0.0761	1.1689	7.9248	18.3438	35.4801	69.2046	81.0601	130.3021	170.5743	243.4506	324.0866
Speedup	10.55	10.34	3.44	3.11	2.70	2.55	2.42	2.99	2.53	2.79	2.92

Cuadro 2: Tiempos y speedup del binario MPI con 2 procesos distribuidos en el clúster.

5.3. MPI Point-to-Point — 4 procesos

MPI Point-to-Point — 4 procesos											
Dim	504	1008	1296	1608	2004	2304	2604	3000	3300	3600	4008
1	0.0554	0.5946	3.9423	8.8963	18.2499	35.1650	40.1974	66.8508	86.5083	120.7626	161.7219
2	0.0650	0.5970	3.8523	9.0573	18.1084	35.0290	39.8755	65.4085	86.3313	121.4657	163.3144
3	0.0647	0.5910	4.1355	9.1250	17.8460	35.2893	40.1333	66.4733	85.6070	123.0662	162.8193
4	0.0654	0.5912	2.2866	8.8631	18.1324	35.4011	41.0048	65.0372	86.5912	122.1995	163.9298
5	0.0661	0.5859	4.3168	9.3068	18.2486	35.9183	40.0194	65.4405	85.5784	121.8314	161.8108
6	0.0646	0.5761	3.9591	8.5487	17.5297	34.1021	40.6271	66.1973	85.1920	121.5897	161.7428
7	0.0557	0.5813	3.9727	8.8045	17.9466	34.5421	40.3667	66.0786	86.6480	121.6881	162.1290
Pr	0.0624	0.5882	3.7807	8.9431	18.0088	35.0638	40.3177	65.9266	86.0652	121.8005	162.4954
Speedup	12.86	20.52	7.21	6.38	5.31	5.03	4.86	5.92	5.02	5.57	5.81

Cuadro 3: Tiempos y speedup del binario MPI con 4 procesos distribuidos en el clúster.

5.4. MPI Point-to-Point — 6 procesos

MPI Point-to-Point — 6 procesos											
Dim	504	1008	1296	1608	2004	2304	2604	3000	3300	3600	4008
1	0.0771	0.4203	2.6642	5.9893	11.9676	23.7609	27.5135	43.3428	57.8124	80.4054	107.8654
2	0.0763	0.4353	2.7624	5.9362	11.8902	23.3436	26.7608	43.5978	57.4197	80.6308	108.3422
3	0.0661	0.4095	2.6407	6.0960	11.9231	23.3406	26.7818	43.6466	56.6631	81.6137	108.3984
4	0.0677	0.4031	2.6151	6.1003	11.6947	23.6634	26.7407	43.9491	57.6445	81.0214	108.4100
5	0.0667	0.4203	2.4444	6.1089	11.9417	23.9487	26.9129	43.6005	58.1408	82.3627	108.8173
6	0.0768	0.4196	2.8927	5.8542	11.9727	23.5043	27.0200	43.5848	57.6752	81.2825	108.1142
7	0.0662	0.4128	2.7499	5.9382	12.2013	23.3337	27.1920	43.5284	57.3770	81.4664	107.5994
Pr	0.0710	0.4173	2.6813	6.0033	11.9416	23.5564	26.9888	43.6071	57.5333	81.2547	108.2210
Speedup	11.31	28.93	10.17	9.50	8.01	7.49	7.26	8.95	7.50	8.35	8.73

Cuadro 4: Tiempos y speedup del binario MPI con 6 procesos distribuidos en el clúster (2 procesos por nodo).

6. Gráficas comparativas

6.1. Tiempo de ejecución

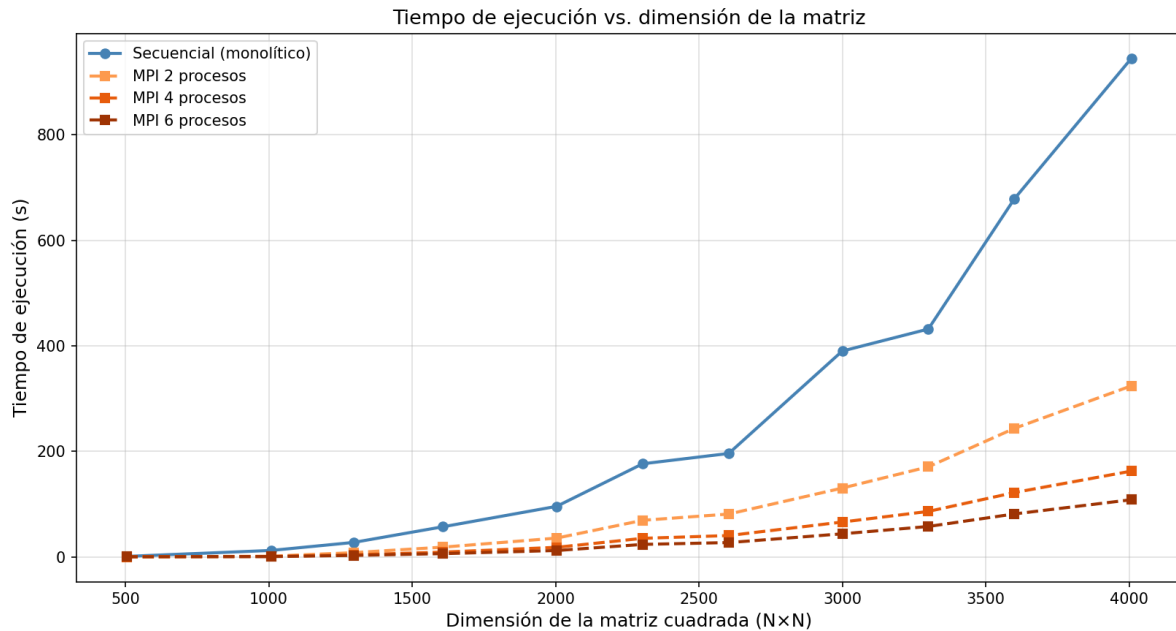


Figura 6: Tiempo de ejecución en función de la dimensión de la matriz para el binario secuencial y las tres configuraciones MPI.

La gráfica de tiempo muestra cómo crece el costo del producto matricial al aumentar N . La curva secuencial domina ampliamente a las paralelas y diverge rápidamente: para $N = 4008$ el secuencial supera los 940 segundos, mientras que la versión con 6 procesos se mantiene por debajo de los 110 segundos. Las tres curvas paralelas presentan la misma forma cúbica del secuencial, pero con una pendiente mucho menor que depende del número de procesos.

6.2. Speedup

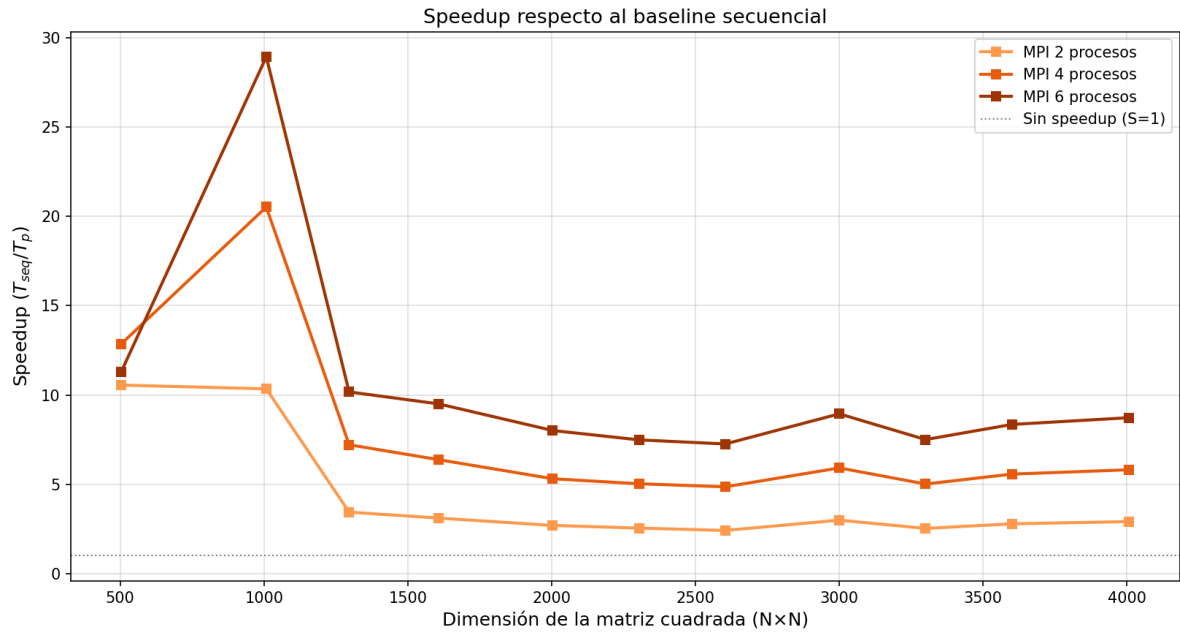


Figura 7: Speedup respecto al baseline secuencial monolítico, en función de la dimensión de la matriz, para 2, 4 y 6 procesos MPI.

La gráfica de speedup es la herramienta más directa para evaluar la ganancia obtenida con MPI. Dos observaciones destacan:

1. Para las matrices más pequeñas (504 y 1008) las tres curvas presentan un pico marcado, llegando a valores entre $10\times$ y $29\times$. Estos valores no son representativos de una ganancia “pura” de paralelización, sino del costo de inicialización del binario secuencial (alocación y llenado de matrices) frente al MPI, donde el grueso del tiempo es comunicación. Se incluyen para no descartar datos, pero el análisis se concentra en las dimensiones grandes.
2. A partir de $N \geq 1296$ las tres curvas se estabilizan claramente: 2 procesos rondan un speedup de $2,5\text{--}3,4\times$, 4 procesos entre $4,9$ y $7,2\times$, y 6 procesos entre $7,3$ y $10,2\times$. La separación entre las tres curvas es consistente y refleja la ganancia real del paralelismo.

6.3. Heatmap de speedup

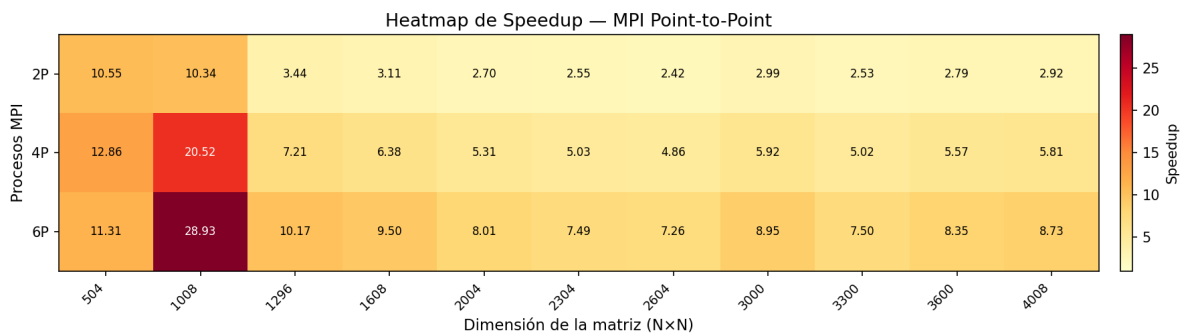


Figura 8: Heatmap de speedup: cada celda muestra S_p para una combinación de configuración (filas) y dimensión de matriz (columnas).

El heatmap resume en una sola vista toda la matriz de resultados. Las zonas más oscuras corresponden a las celdas con speedup más alto y se concentran principalmente en la combinación de 6 procesos +

matrices entre 1296 y 4008. La “zona estable” (a partir de $N \geq 1296$) muestra un patrón ordenado donde el speedup crece monótonamente con el número de procesos, lo que es coherente con la teoría: a mayor cómputo total, mejor amortización del overhead de comunicación.

7. Análisis de Resultados

7.1. Comportamiento del speedup observado

Para matrices grandes ($N \geq 2000$), los speedups observados son consistentes y crecen con el número de procesos. Tomando $N = 4008$ como referencia, los valores son: $S_2 = 2,92$, $S_4 = 5,81$ y $S_6 = 8,73$. Las relaciones entre configuraciones MPI son particularmente informativas: pasar de 2 a 4 procesos aporta un factor de $5,81/2,92 \approx 1,99\times$, casi exactamente el factor 2 esperado al duplicar recursos. De 4 a 6 procesos se obtiene un factor $8,73/5,81 \approx 1,50\times$, también muy cercano a la relación ideal $6/4 = 1,50$. Esto sugiere que el algoritmo está aprovechando casi por completo los recursos adicionales del clúster.

7.2. Picos en matrices pequeñas

Para $N \in \{504, 1008\}$ el speedup absoluto supera el número de procesos disponibles (hasta $29\times$ con 6 procesos sobre $N = 1008$). Este fenómeno, conocido como *speedup superlineal*, suele explicarse por dos factores: (i) los costos fijos de inicialización del binario secuencial (asignación, llenado de matrices con `rand()` y liberación de memoria) representan una fracción comparativamente alta del tiempo para matrices pequeñas; y (ii) cuando el problema se reparte entre procesos, cada porción puede caber en niveles más rápidos de la jerarquía de memoria de cada nodo, mejorando la utilización de caché por proceso. Para los tamaños grandes el cómputo domina y la curva “aterrija” en valores razonables.

7.3. Impacto del overhead de comunicación

En MPI, antes de comenzar a calcular, el rank 0 debe enviar a cada worker su batch de filas de *A* más la matriz *B* completa (tag 1). Este último envío es proporcional a n^2 enteros por cada worker, lo que para $N = 4008$ representa cerca de 64MB transferidos por la red por cada worker. A pesar de este costo, los nodos están en la misma subred (latencia intra-AZ) y el ancho de banda intra-AWS basta para que el envío se amortice contra los $\mathcal{O}(n^3)$ operaciones de cómputo asignadas a cada worker. En matrices muy pequeñas esta amortización falla: el tiempo de comunicación es comparable al tiempo de cómputo y el escalado se desvía del ideal.

7.4. Configuración de Slurm y techo de paralelismo

La asignación `salloc --nodes=3 --ntasks=6 --ntasks-per-node=2` fija el techo en 6 procesos MPI, exactamente igual al número total de vCPUs del clúster (3 nodos $\times$ 2 vCPUs/nodo). Intentar lanzar más procesos sin overcommit requeriría: (i) aumentar el número de nodos (`MaxCount`), o (ii) cambiar el tipo de instancia a uno con más vCPUs por nodo (por ejemplo `c5.xlarge` con 4 vCPUs). El comportamiento casi lineal observado hasta $p = 6$ sugiere que ampliar la cota probablemente seguiría aportando speedup adicional para problemas suficientemente grandes.

7.5. Comparación cualitativa con paralelismo de memoria compartida

En el Caso de Estudio 1 se evaluaron implementaciones de memoria compartida (hilos POSIX y `fork` con `mmap`) sobre un Apple M4 de 10 núcleos. Aquellas variantes no tienen costo explícito de transferencia de datos porque los workers leen las matrices directamente de memoria compartida, y por eso alcanzaban speedups cercanos al ideal con menor overhead. La implementación MPI, en cambio, paga el costo de comunicación por la red, pero a cambio puede agregar memoria y cómputo de varias máquinas independientes, lo que permite escalar problemas que no caben en una sola. Para este caso (matriz 4008×4008 , ≈ 64 MB por matriz) el clúster cumple la promesa: la versión MPI con 6 procesos no es más rápida que un Apple M4 con 8–16 hilos, pero demuestra que la arquitectura distribuida es viable y que el costo de comunicación es manejable cuando la relación cómputo/comunicación es favorable.

8. Conclusiones

1. **Escalabilidad casi lineal entre configuraciones MPI.** Para matrices $N \geq 2000$, duplicar el número de procesos (de 2 a 4) duplica el speedup, y pasar a 6 procesos lo aumenta en un factor cercano a $1,5\times$, ambos coherentes con la relación ideal entre recursos. El algoritmo punto a punto implementado aprovecha los nodos del clúster de manera efectiva dentro del rango de procesos disponible.
2. **El overhead de comunicación domina en matrices pequeñas.** Para $N \leq 1296$ los tiempos de envío de A y B por la red son comparables al tiempo de cómputo, y el speedup observado no es representativo del paralelismo real. La paralelización distribuida sólo compensa para problemas suficientemente grandes.
3. **Configuración óptima dentro del clúster: 6 procesos en 3 nodos.** Con la configuración `c7i-flex.large` y `ntasks-per-node=2`, el techo natural es 6 procesos. En esa configuración se alcanza $S_6 = 8,73$ para $N = 4008$ respecto al baseline secuencial monolítico. Para escalar más allá habría que cambiar el tipo de instancia o aumentar el número de nodos.
4. **AWS ParallelCluster reduce la complejidad operacional pero introduce ataduras.** El despliegue requirió Python 3.9 específicamente, par de claves SSH, configuración YAML, manejo de subredes y revisión del estado del stack en CloudFormation. A cambio, una vez en funcionamiento, Slurm y EFS permiten ejecutar el benchmark exactamente como se haría en un clúster on-premise, sin necesidad de scripts manuales de distribución de binarios.
5. **EFS resuelve la distribución del binario, pero no la del dato.** El sistema de archivos compartido elimina la fricción del despliegue (basta con `make` en el HeadNode), pero la matriz B debe seguir siendo enviada explícitamente a cada worker en cada corrida. Una optimización futura sería leer B directamente del EFS desde cada worker, evitando la transferencia por MPI.
6. **MPI_Wtime resuelve la advertencia del enunciado sobre relojes heterogéneos.** Toda la medición ocurre desde el rank 0, sobre un único reloj, evitando comparar tiempos entre nodos con tasas de avance diferentes.

9. Bibliografía

Referencias

- [1] Cortéz, A. (2004). Teoría de la complejidad computacional y teoría de la información. *Revista de Investigación de Sistemas e Informática*, 1(1), 102–105. <https://revistasinvestigacion.unmsm.edu.pe/index.php/sistem/article/view/3216>
- [2] Ferestrepoca. (2019). Paradigmas de programación paralela. http://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teoría/index.html
- [3] López, A. (2017). Introducción al High Performance Computing. *Encamina Blog*. <https://blogs.encamina.com/por-una-nube-sostenible/introduccion-al-high-performance-computing/>
- [4] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of AFIPS Spring Joint Computer Conference*, pp. 483–485.
- [5] MPI Forum. (2023). *MPI: A Message-Passing Interface Standard*, Version 5.0. <https://www.mpi-forum.org/>
- [6] The Open MPI Project. (2024). *Open MPI: Open Source High Performance Computing*. <https://www.open-mpi.org/>
- [7] Kendall, W. (2014). *MPI Tutorial*. <https://github.com/mpitutorial/mpitutorial>
- [8] Amazon Web Services. (2024). *AWS ParallelCluster User Guide*. <https://docs.aws.amazon.com/parallelcluster/>